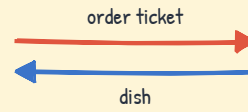
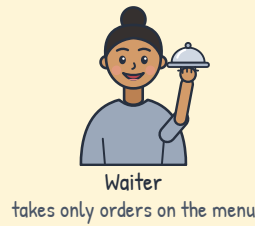
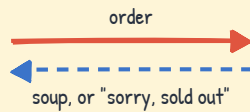


6. REST API: ORDER FROM THE MENU, NOT THE KITCHEN

ANALOGY: A RESTAURANT

1 GUEST ORDERS, KITCHEN COOKS

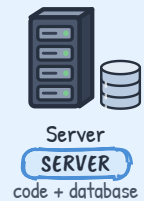
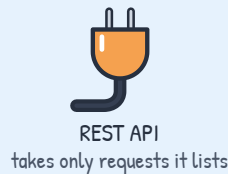
One soup, please.



2 THE SAME IN SOFTWARE



Your app
CLIENT
reads the API docs



The API is the menu: the requests a server accepts. Clients order from it, never walk into the kitchen.

THE THING AND THE VERB

- URL** • Names the thing: `/orders` is the list, `/orders/42` is one order.
- GET** • Read. `GET /orders/42` asks what is in order 42. Nothing changes.
- POST** • Create. `POST /orders` with a body adds a new order. The server picks its number.
- PUT** • Replace. `PUT /orders/42` sends the whole order as it should look now.
- PATCH** • Change a part. `PATCH /orders/42` with `{"dish": "salad"}` swaps only the dish.
- DELETE** • Remove. `DELETE /orders/42` cancels the order.

HOUSE RULES



SAFE

GET only looks. Asking what you ordered never changes the order.



IDEMPOTENT

Twice ends like once. GET, PUT and DELETE are; POST is not: two POSTs, two soups.



STATELESS

The waiter forgets you. Each request carries all it needs, even your token.

STATUS: 2XX WORKED, 4XX CLIENT, 5XX SERVER



200
OK

Here is your dish: the data you asked for.



201
Created

New order taken, usually after POST.



204
No Content

Done, nothing to bring back. Common after DELETE.



400
Bad Request

The order makes no sense: broken JSON.



401
Unauthorized

Despite the name: we do not know who you are.



403
Forbidden

We know you, but you may not do this.



404
Not Found

Not on the menu: no such order.



405
Method Not Allowed

This verb is not on the menu for this URL.



429
Too Many Requests

Too many orders too fast. Wait, then retry.



500
Internal Server Error

The kitchen broke. Not your fault.

ORDER FROM A TERMINAL (SAMPLE)

```
$ curl -i -X POST https://api.example.com/orders \
-H "Content-Type: application/json" \
-d '{"dish": "soup"}'
HTTP/2 201 ...headers...
{"id":42,"dish":"soup"}
api.example.com is a placeholder. -i shows the status line; without it you see only the body.
```



REMEMBER: Order from the menu: the URL names the thing, the method says what to do, and the status code tells you how it went.